

软件构造

-----第4章 软件单元测试

主讲：彭辉

Email: penghui@cuit.edu.cn

第4章 软件单元测试

4.1 软件测试基础

4.2 单元测试

4.3 白盒测试技术

4.4 黑盒测试技术

4.5 单元测试用例书

4.1 软件测试基础

什么是软件测试？

是为了发现错误而执行程序的过程。

发现错误是为了更正错误，最终得到一个高质量的软件系统。

软件测试的对象：整个软件定义、开发周期的产品。软件测试不等于程序测试；它包括程序测试和文档测试。

测试用例：通常指测试数据和预期的输出结果

4.1.1 软件测试的目标

软件测试存在的矛盾

用户希望通过软件测试暴露软件中隐藏的错误和缺陷，以考虑是否可接受该产品。

软件开发者希望通过软件测试表明软件产品中不存在错误，已正确地实现了用户的要求。

G.Myers给出的软件测试目的

- 测试是为了发现程序中的错误而执行程序的过程
- 好的测试用例是极可能发现迄今为止尚未发现的错误的测试方案；
- 成功的测试是发现了至今未发现的错误的测试

总之，测试的目的是以最少的时间和人力，系统地找出软件中潜在的各种错误和缺陷；测试附带的收获是它能证明软件的功能和性能与需求说明相符合。

注意：测试不能表明软件中不存在错误，它只能说明软件中存在错误。

4.1.2 软件测试的准则

- 1) 所有测试都能追溯到用户需求
- 2) 应该远在测试开始之前就制定出测试计划
- 3) 应该把 **Pareto**原理（又叫**80/20**定律）应用到软件测试中。

- ❑ 一百多年前一位名叫**Vilfredo Pareto**的意大利经济学家研究了财富在人口中的分布情况。他发现大部分财富都在较少数人的手中,也就是说财富的分布是不均等的。
- ❑ **80 %**的错误可能是由 **20 %**的模块造成的
- ❑ 可以用**20%**的工作量得到**80%**的结果

4) 从“小规模”测试开始，逐步过渡到“大规模”测试

5) 穷举测试是不可能的

□ 测试只能证明程序有错，不能证明程序没有错误

6) 应由独立的第三方从事测试工作

4.1.3 测试方法

软件测试方法一般分为：静态测试和动态测试。

静态测试是指被测程序不在机器上运行，采用人工检测和计算机辅助静态分析的手段对程序进行检测。

动态测试是指通过运行程序发现错误，又分**黑盒法**和**白盒法**两种。

两种动态测试方法

黑盒测试(功能测试):

- 把程序看作一个黑盒子;
- 完全不考虑程序的内部结构和处理过程;
- 是在程序接口进行的测试。

白盒测试(结构测试):

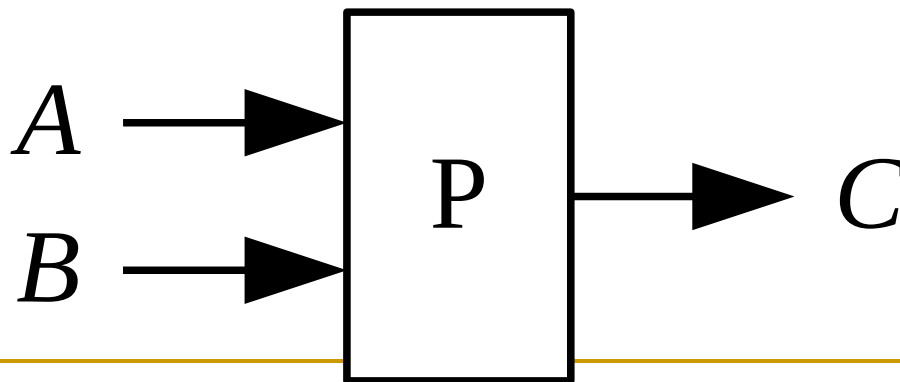
- 把程序看成装在一个透明的盒子里;
- 测试者完全知道程序的结构和处理算法;
- 按照程序内部的逻辑测试程序, 检测程序中的主要执行通路是否都能按预定要求正确工作。

黑盒测试不可能实现穷尽测试：

假设有程序P，输入量为A和B，输出量为C。

如果计算机的字长为32位，A和B的数据类型都是整数类型。将A和B的可能取值进行排列组合，输入数据的可能性有： $2^{32} \times 2^{32} = 2^{64}$ 种。

假设这个程序执行一次需要1毫秒，要完成所有的测试，计算机需要连续工作5亿年。

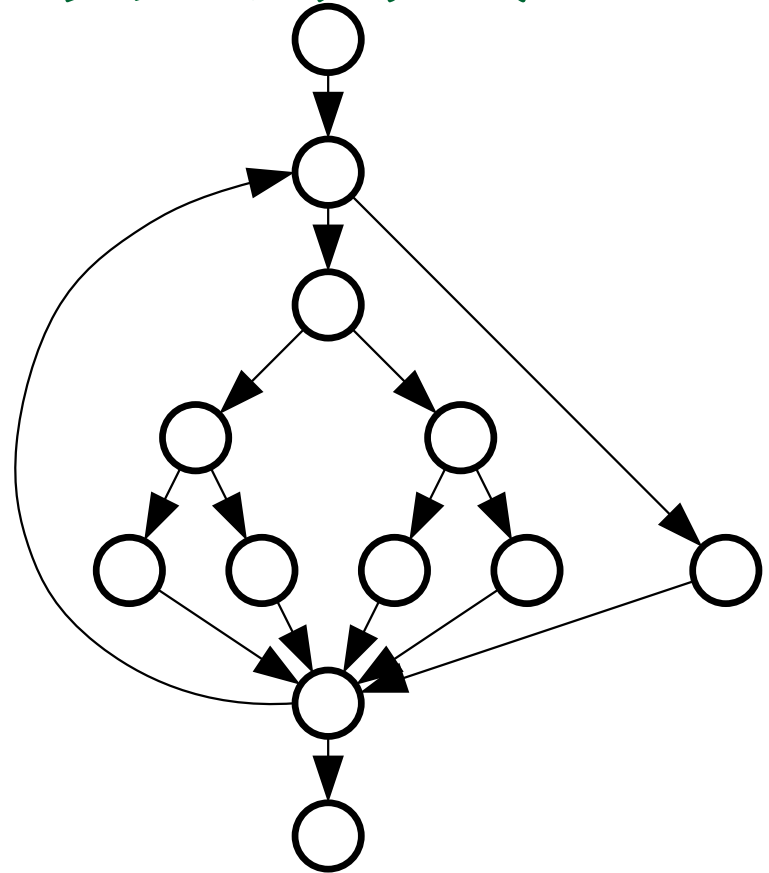


白盒测试也不能实现穷尽测试

图中所示的一个小程序的控制流程。曲线代表执行次数不超过20的循环，循环体中共有5条通路。

可能执行的路径有 5^{20} 条，近似为 10^{14} 条可能的路径。

如果完成一个路径的测试需要1毫秒，那么整个测试过程需要3170年。



黑盒测试与白盒测试优缺点比较:

	黑盒测试	白盒测试
优点	①适用于各阶段测试 ②从产品功能角度测试 ③容易入手生成测试数据	①通常在单元测试阶段使用 ②有一定的充分性度量手段
缺点	①某些代码得不到测试 ②如果规格说明有误，则无法发现 ③不易进行充分性测试	①通常不易生成测试数据 ②无法对未实现规格说明的部分进行测试 ③工作量大，有应用局限
性质	一种确认技术，回答“我们在构造一个正确的系统吗？”	一种验证技术，回答“我们在正确地构造一个系统吗？”

4.1.4 测试步骤

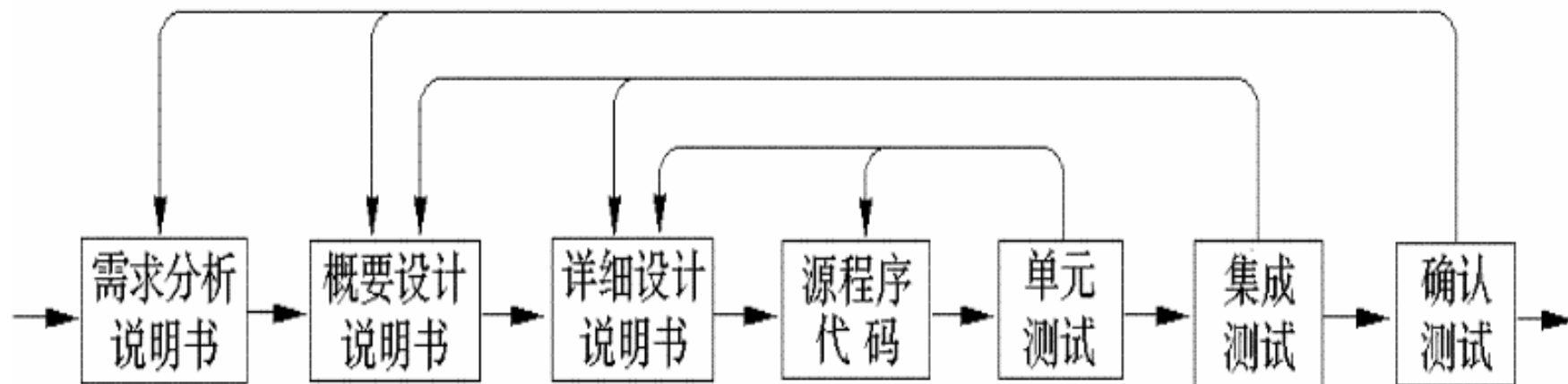
测试的 5 个步骤：

- 单元（模块）测试
- 集成测试
- 系统测试
- 验收测试（确认测试）
- 上线测试（平行运行测试）

测试与软件开发各阶段的关系

软件开发过程是一个自顶向下，逐步细化的过程

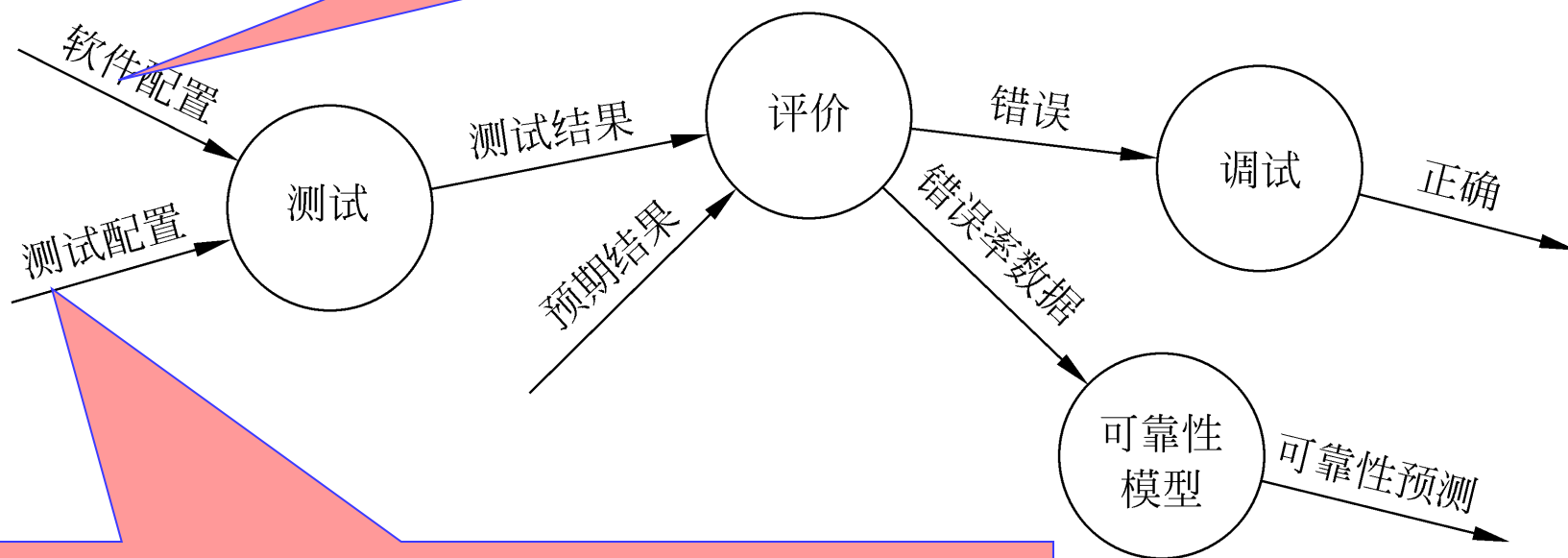
软件测试则是依相反顺序自底向上，逐步集成的过程。



4.1.5 测试阶段的信息流

测试过程需要二类输入

软件配置：软件需求规格说明、软件设计规格说明、源代码等；



测试配置：测试计划、测试用例、测试程序等；

4.2 单元测试（Unit Testing）

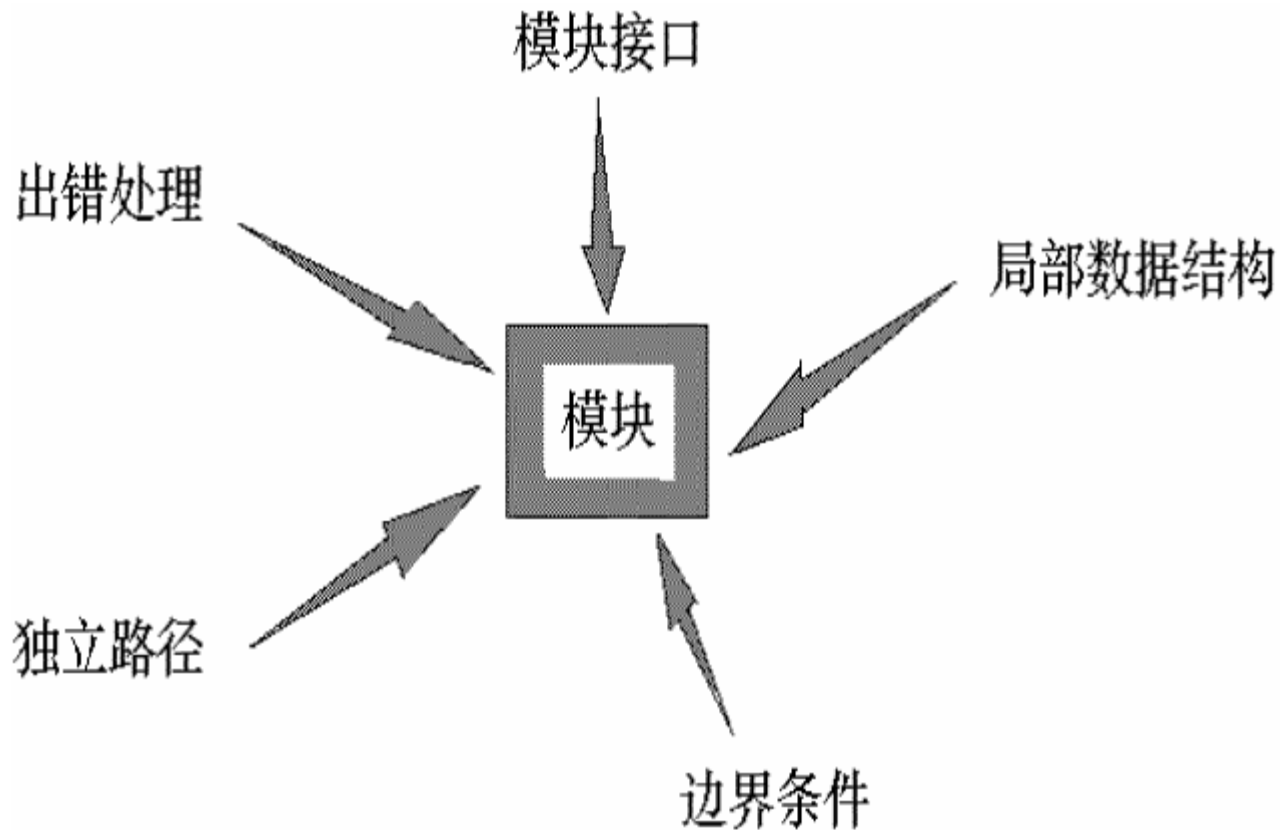
又称模块测试，需要从程序的内部结构出发设计测试用例。

测试目的：发现模块内部可能存在的差错

测试依据：详细设计说明书和源程序清单

测试方法：白盒测试为主，黑盒测试为辅，多个模块并行进行。

4.2.1 测试重点



1) 模块接口测试

在单元测试的开始，应对被测模块的接口进行测试。

测试项目：

- ❑ 调用本模块的输入参数是否正确；
- ❑ 本模块调用子模块时，输入给子模块的参数是否正确；
- ❑ 输出参数是否正确；
- ❑ 与外部设备的输入输出是否正确。

2) 局部数据结构测试

测试项目：

- ❑ 不正确或不一致的数据类型说明
- ❑ 使用尚未赋值或尚未初始化的变量
- ❑ 错误的初始值或错误的缺省值
- ❑ 变量名拼写错或书写错
- ❑ 不一致的数据类型

3) 重要的执行通路测试

由于不能穷尽测试，选择最有代表性、最可能发现错误的执行通路进行测试。关键：测试用例要适当。

对运行时间有要求的模块，还要专门进行关键路径测试，以确定最坏情况下和平均意义下影响模块运行时间的因素。

4) 错误处理测试

着重测试以下可能发生的错误:

- ❑ 对出错的描述是难以理解的
- ❑ 显示的错误与实际的错误不同
- ❑ 对错误处理不正确
- ❑ 描述错误的信息不足以帮助确定造成错误的位置

5) 边界测试

重点检查刚好等于、大于或小于边界值的数据;

4.2.2 代码审查

人工测试源程序可以由程序编写者本人非正式进行，也可由审查小组正式进行，后者称为代码审查。对于典型的程序来说，可以查出30%-70%的逻辑设计错误和编码错误。

参与者：组长（没有直接参与系统开发，但有能力的程序员。）、程序的设计者、编写者、测试者。

方法：研究设计说明书，一起审查程序代码如何实现设计，从中发现问题。

注意：通常代码审查和机器测试结合使用。

4.2.3 计算机测试

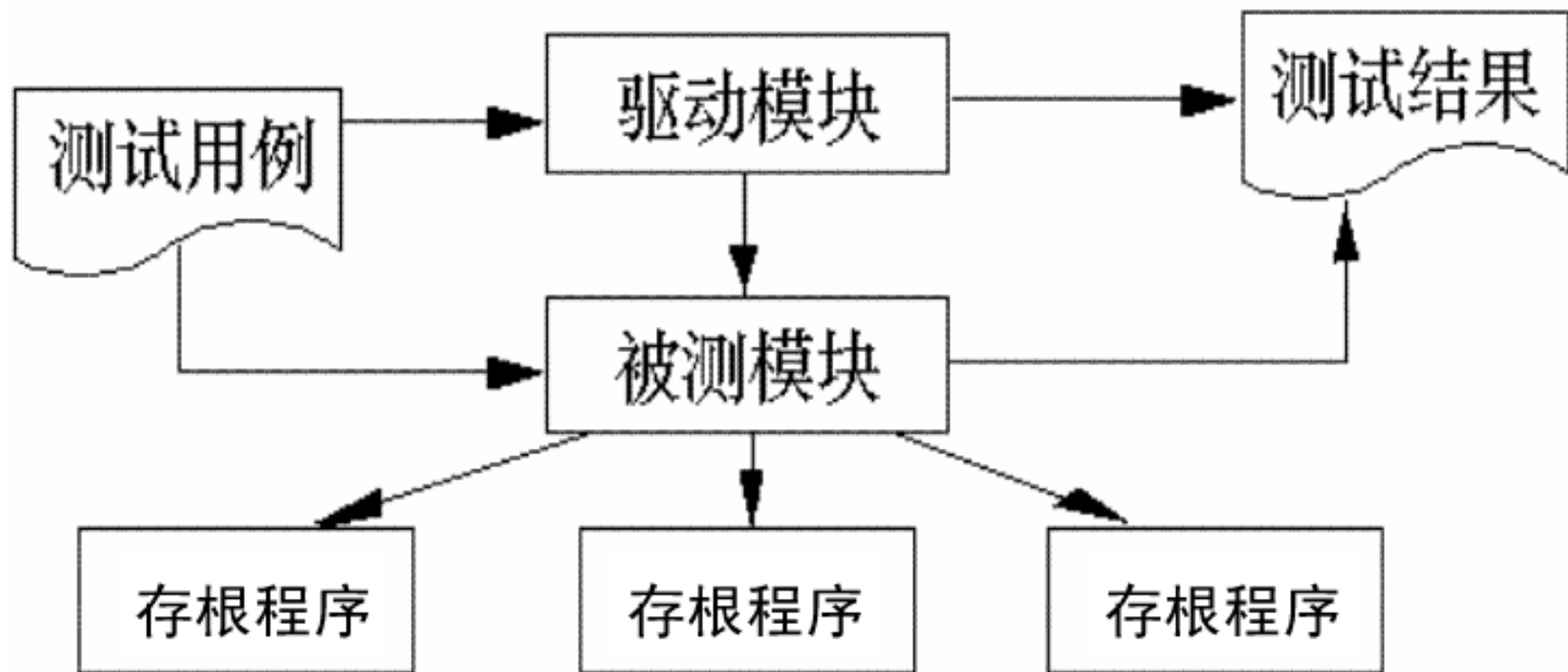
单元测试通常在编码阶段进行。常用**机器测试**，即通过运行模块发现问题。

两个重要概念：

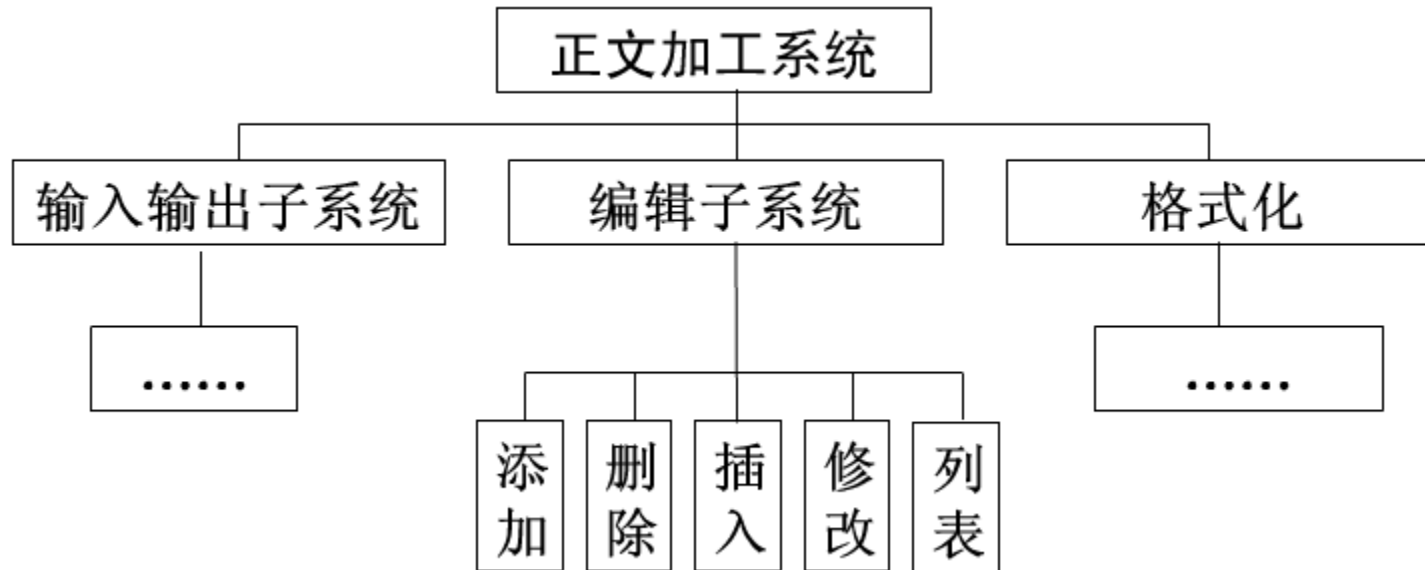
驱动程序（driver）：相当于被测试模块的“主程序”，接收测试数据，把这些数据传送给被测试的模块，并且输出相关结果。

存根程序（stub）：又称为**桩模块**，代替被测试模块**所调用**的模块。不需要具有子模块所有功能，但不允许什么事情也不做。

单元测试的测试环境



正文加工系统功能结构图



若要测试“编辑子系统”，需要上层“驱动模块”和下层“存根程序”

4.3 白盒测试技术

白盒测试执行的要求：

对程序模块的所有独立的执行路径至少测试一次。

在循环的边界和运行界限内执行循环体；循环体内的每个路径至少执行一次。

对所有的逻辑判定，取“真”与取“假”的两种情况都至少测试一次；

4.3.1 逻辑覆盖

- 有选择地执行程序某些最有代表性的通路是对穷尽测试的惟一可行的替代办法。
- 从覆盖源程序语句的详尽程度分析，大致有以下一些不同的覆盖标准：
 - 语句覆盖
 - 判定覆盖
 - 条件覆盖
 - 判定/条件覆盖
 - 条件组合覆盖

1. 语句覆盖

含义：选择足够多的测试数据，使被测程序中每个语句至少执行一次。

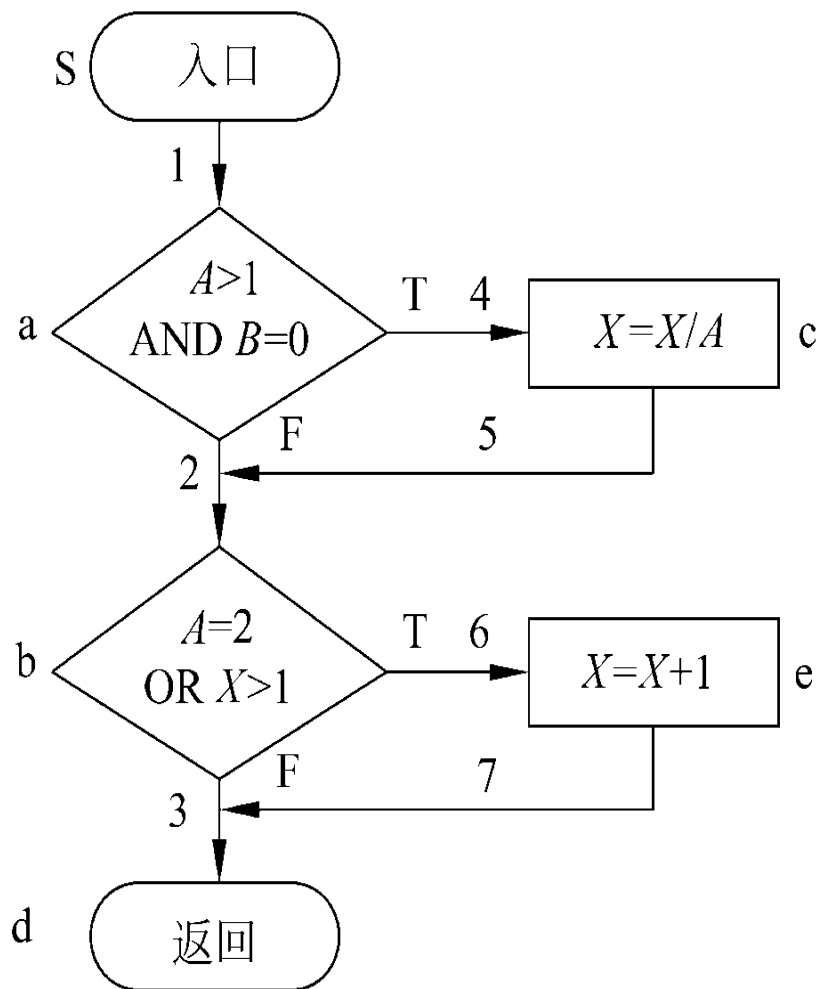
分析：

执行路径sacbed

测试用例：

$A=2$, $B=0$, $X=4$

覆盖sacbed



语句覆盖特点：

语句覆盖对程序的逻辑覆盖很少。

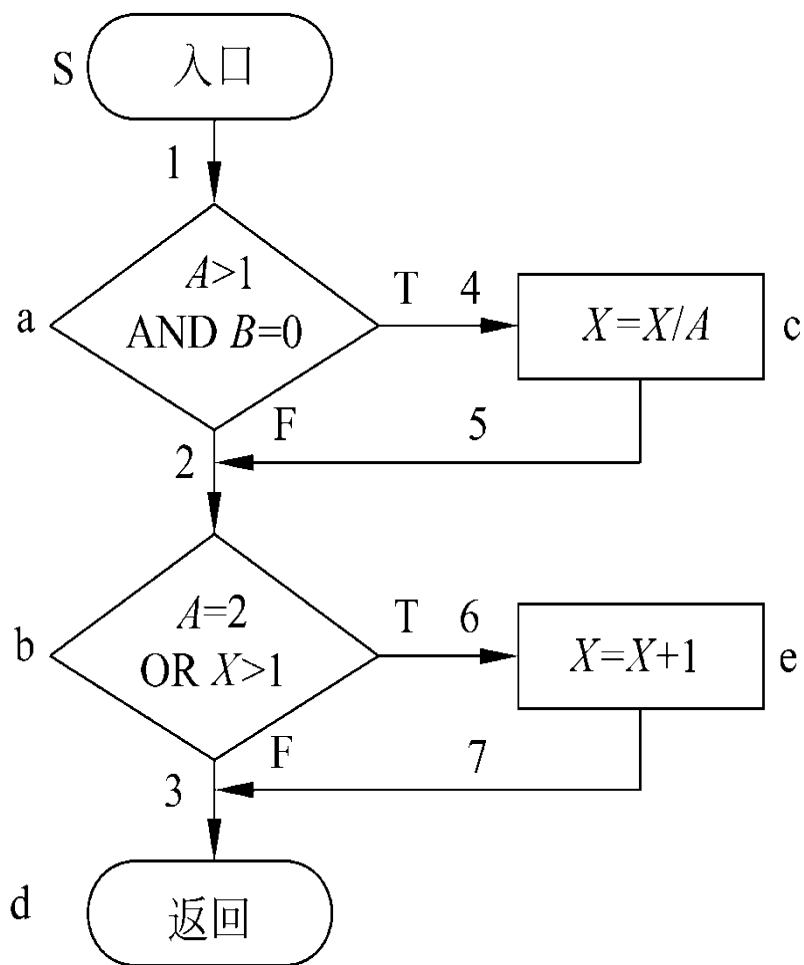
语句覆盖是很弱的逻辑覆盖标准。

2. 判定覆盖（分支覆盖）

含义： 不仅每个语句必须至少执行一次，而且每个判定的每种可能的结果都应该至少执行一次。

所有判定分支：

- (1) a点判定为T
- (2) a点判定为F
- (3) b点判定为T
- (4) b点判定为F



2. 判定覆盖（分支覆盖）

测试用例：

I. 满足(1)(4)

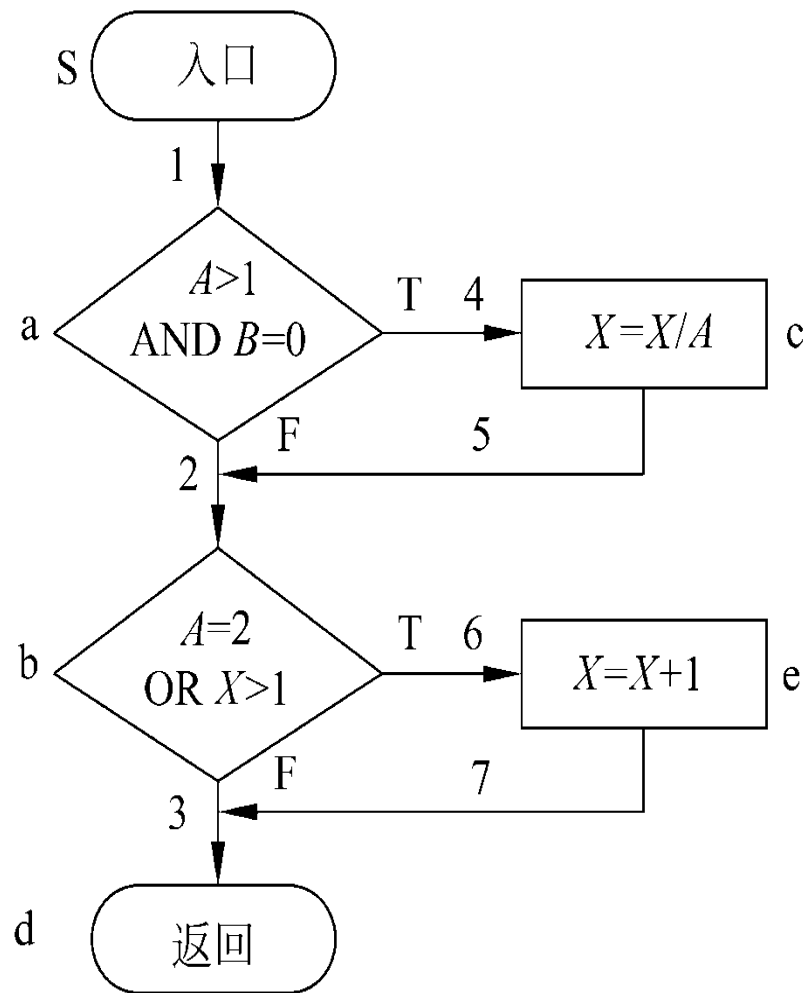
$A=3$, $B=0$, $X=3$

覆盖sacbd

II. 满足(2)(3)

$A=2$, $B=1$, $X=1$

覆盖sabed



2. 判定覆盖（分支覆盖） 或者

I. 满足(1)(3)

$A=2$, $B=0$, $X=3$

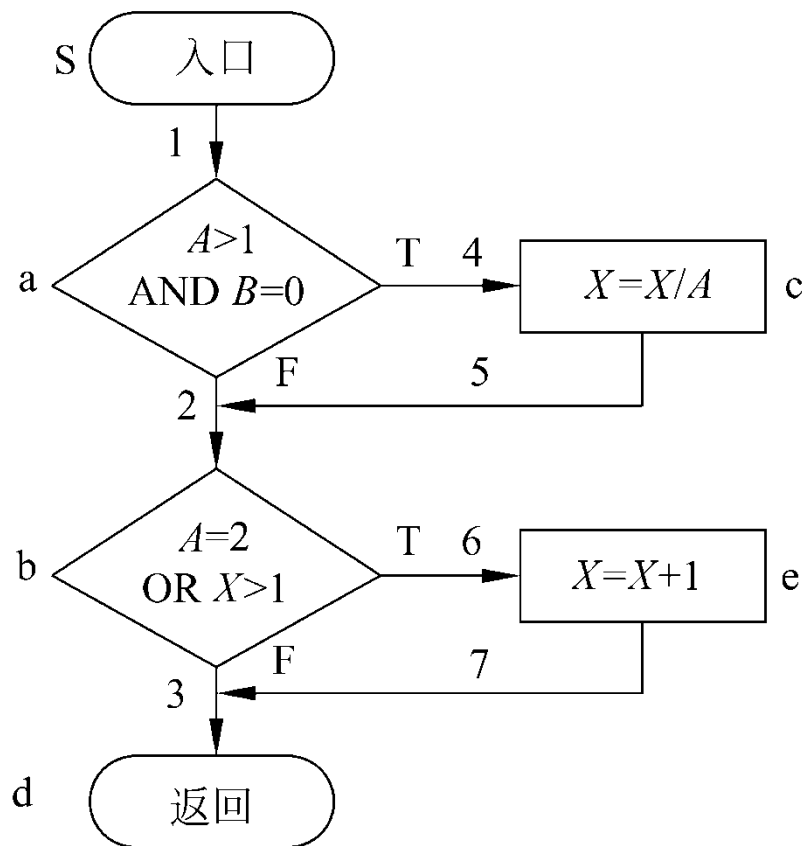
覆盖sacbed

II. 满足(2)(4)

$A=3$, $B=1$, $X=1$

覆盖sabd

特点：比语句覆盖强，但对
程序逻辑的覆盖程度仍不
高。



3. 条件覆盖

含义：不仅每个语句至少执行一次，而且使判定表达式中的每个条件都取到各种可能的结果。

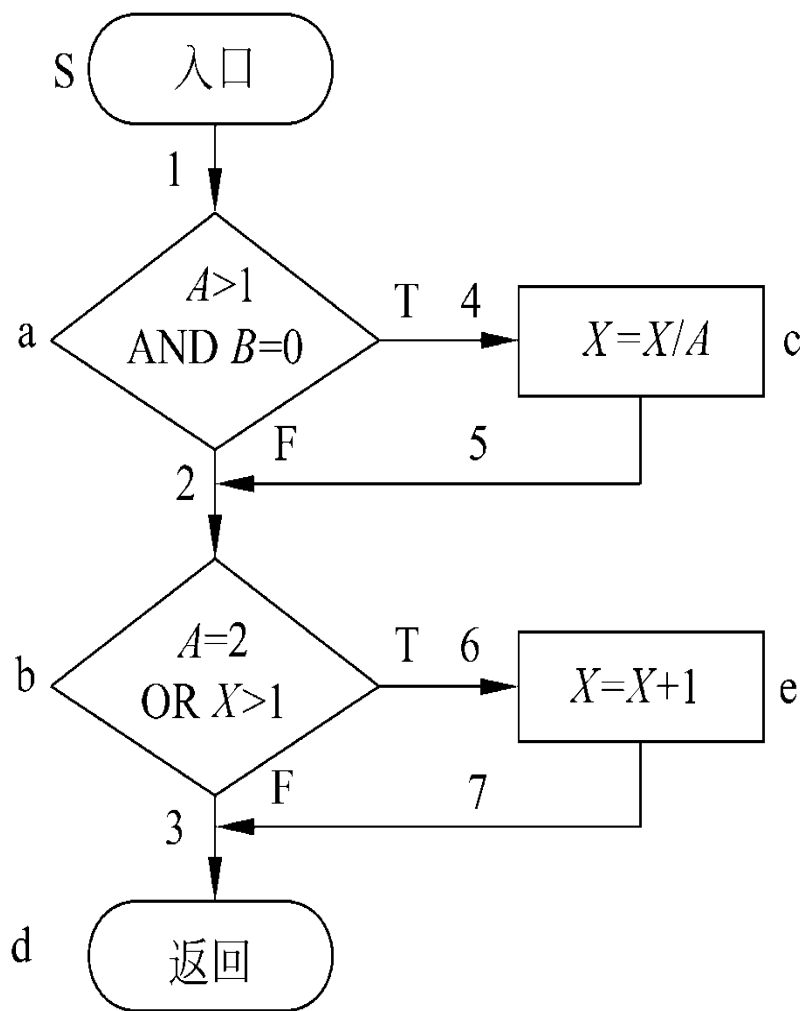
所有条件：

(1) $A > 1$ (2) $A \leq 1$

(3) $B = 0$ (4) $B \neq 0$

(5) $A = 2$ (6) $A \neq 2$

(7) $X > 1$ (8) $X \leq 1$



3. 条件覆盖

测试用例：

I. 满足(1)(3)(5)(7)

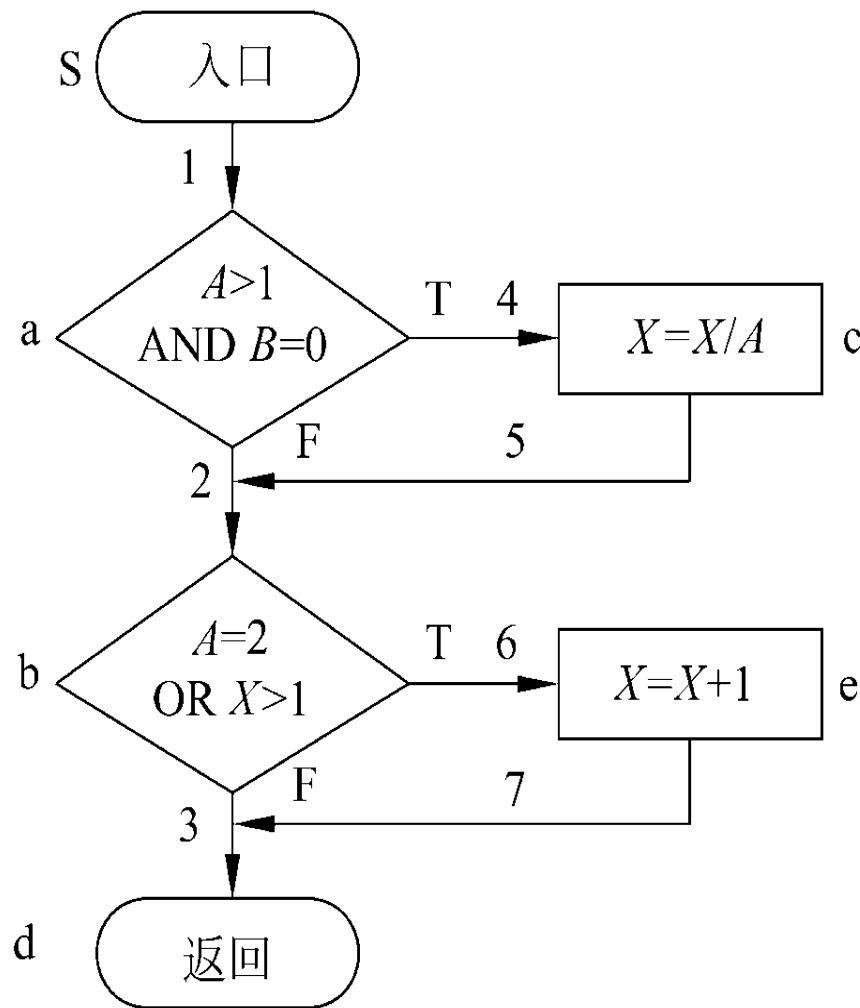
$A=2$, $B=0$, $X=4$

覆盖sacbed

II. 满足(2)(4)(6)(8)

$A=1$, $B=1$, $X=1$

覆盖sabd



3. 条件覆盖 或者

I. 满足(1)(3)(5)(8)

$A=2$, $B=0$, $X=1$

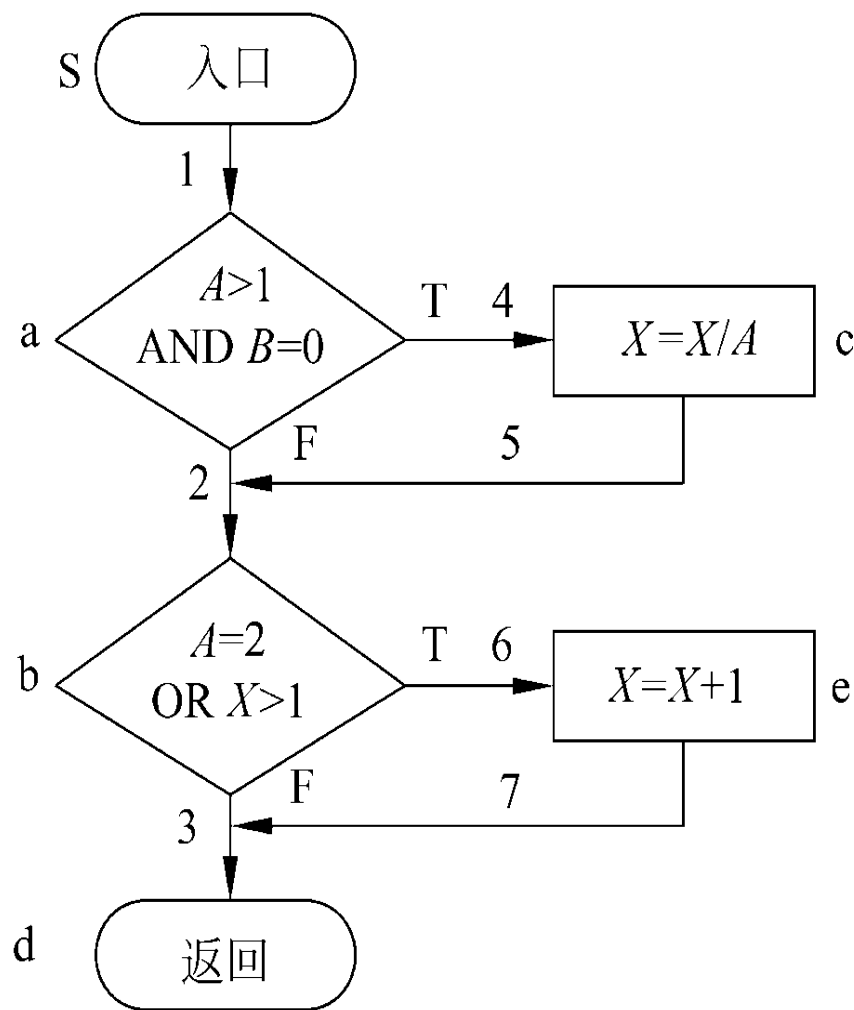
覆盖sacbed

II. 满足(2)(4)(6)(7)

$A=1$, $B=1$, $X=2$

覆盖sabcd

或者.....



条件覆盖特点：

- 条件覆盖通常比判定覆盖强，因为它使每个条件都取到了两个不同的结果，判定覆盖却只关心整个判定表达式的值。但也有反例，如第二组测试用例。
- 判定覆盖不一定包含条件覆盖，条件覆盖也不一定包含判定覆盖。

4. 判定/条件覆盖

含义：使得判定表达式中的每个条件都取到各种可能的值，每个判定表达式也都取到各种可能的结果。

测试用例：

I. 满足条件(1)(3)(5)(7)

和判定(1)(3)

$A=2, B=0, X=4$

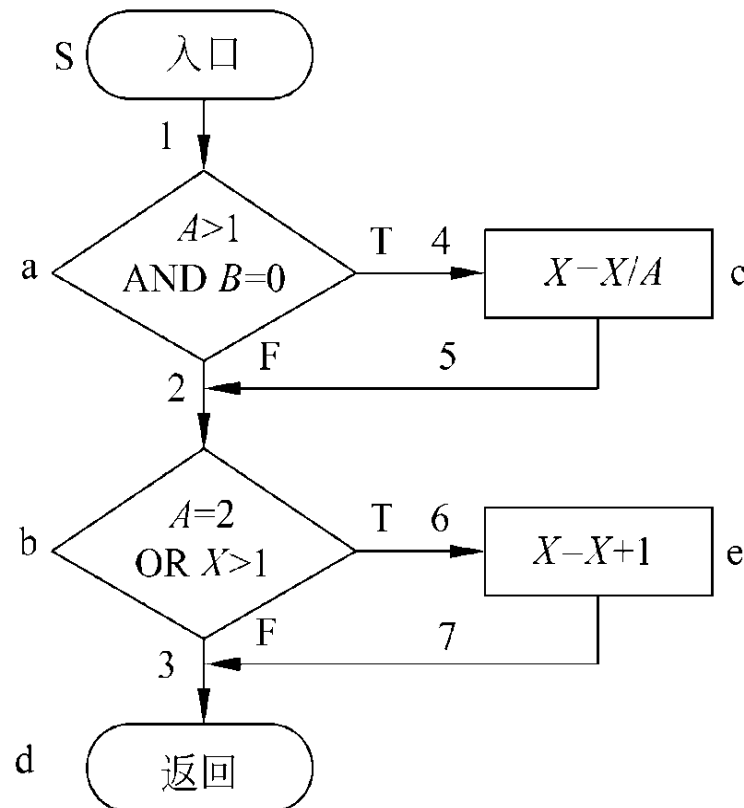
覆盖sacbed

II. 满足条件(2)(4)(6)(8)

和判定(2)(4)

$A=1, B=1, X=1$

覆盖sabd



特点：有时判定/条件覆盖也并不比条件覆盖更强。

5. 条件组合覆盖

含义：要求选取足够多的测试数据，使得每个判定表达式中条件的各种可能组合都至少出现一次。

条件组合：

(1) $A > 1$, $B = 0$

(2) $A > 1$, $B \neq 0$

(3) $A \leq 1$, $B = 0$

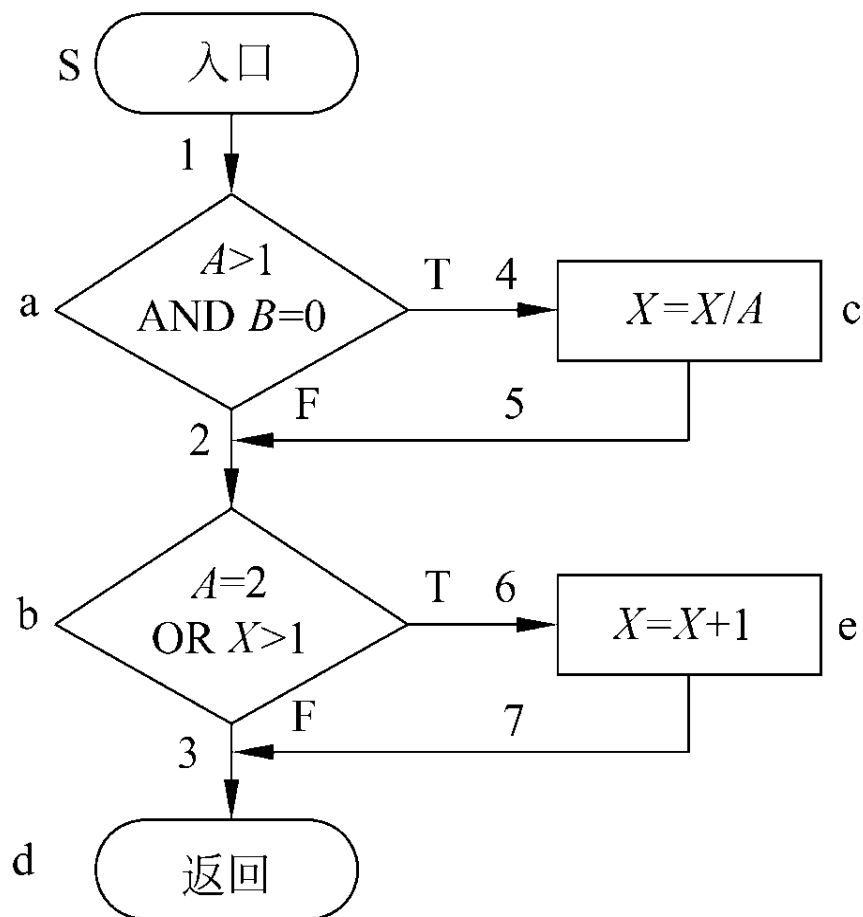
(4) $A \leq 1$, $B \neq 0$

(5) $A = 2$, $X > 1$

(6) $A = 2$, $X \leq 1$

(7) $A \neq 2$, $X > 1$

(8) $A \neq 2$, $X \leq 1$



5. 条件组合覆盖

测试用例：

I. 满足(1)(5)

$A=2$, $B=0$, $X=4$

覆盖sacbed

II. 满足(2)(6)

$A=2$, $B=1$, $X=1$

覆盖sabed

III. 满足(3)(7)

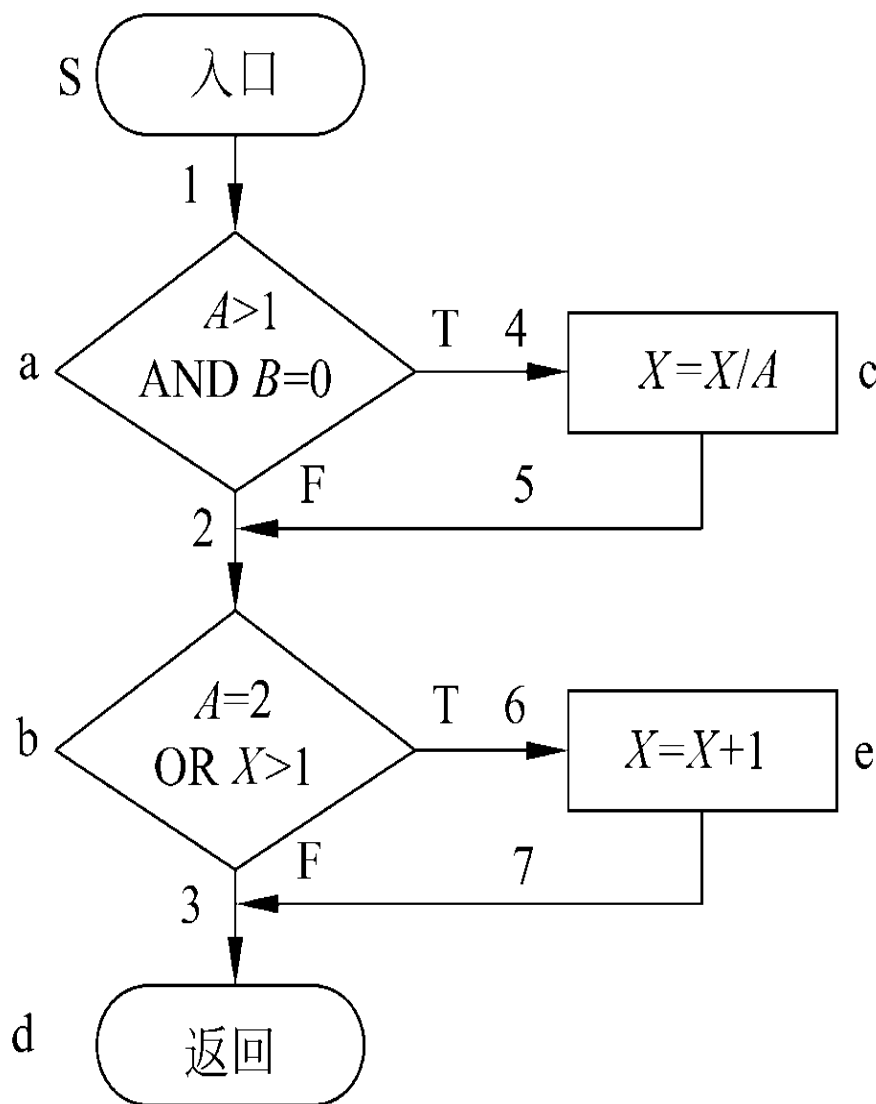
$A=1$, $B=0$, $X=2$

覆盖sabed

IV. 满足(4)(8)

$A=1$, $B=1$, $X=1$

覆盖sabd



条件组合覆盖特点：

- 条件组合覆盖是前述几种覆盖标准中最强的。满足条件组合覆盖标准的测试数据，也一定满足判定覆盖、条件覆盖和判定/条件覆盖标准。
- 但是，条件组合覆盖标准的测试数据并不一定能使程序中的每条路径都执行到。（4组数据都没有测试到sacbd）

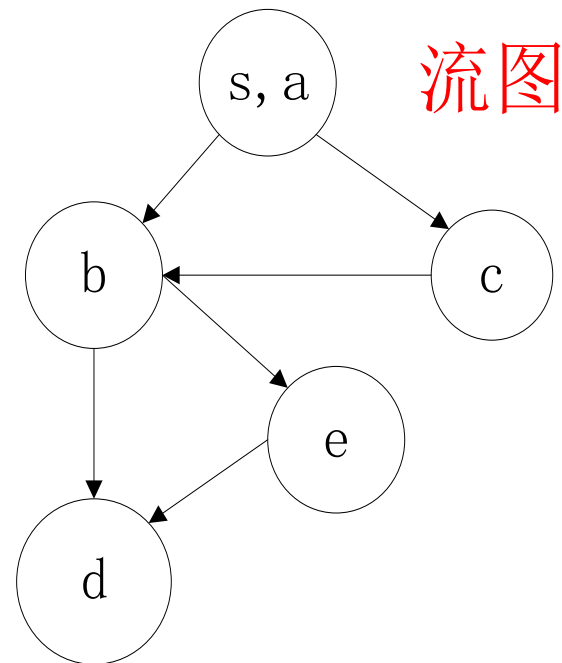
从对程序路径的覆盖程度分析的逻辑覆盖标准：

6. 点覆盖

- 含义：选取足够多的测试数据，使得程序执行路径至少经过流图的每个结点一次。
- 特点：由于流图的每个结点与一条或多条语句相对应，因此点覆盖标准和语句覆盖标准是相同的。

7. 边覆盖

- 含义：选取足够多测试数据，使得程序执行路径至少经过流图中每条边一次。
- 特点：通常边覆盖和判定覆盖是一致的。



8. 路径覆盖

含义：选取足够多测试数据，使程序的每条可能路径都至少执行一次(如果程序图中有环，则要求每个环至少经过一次)。

路径覆盖

条件组合覆盖

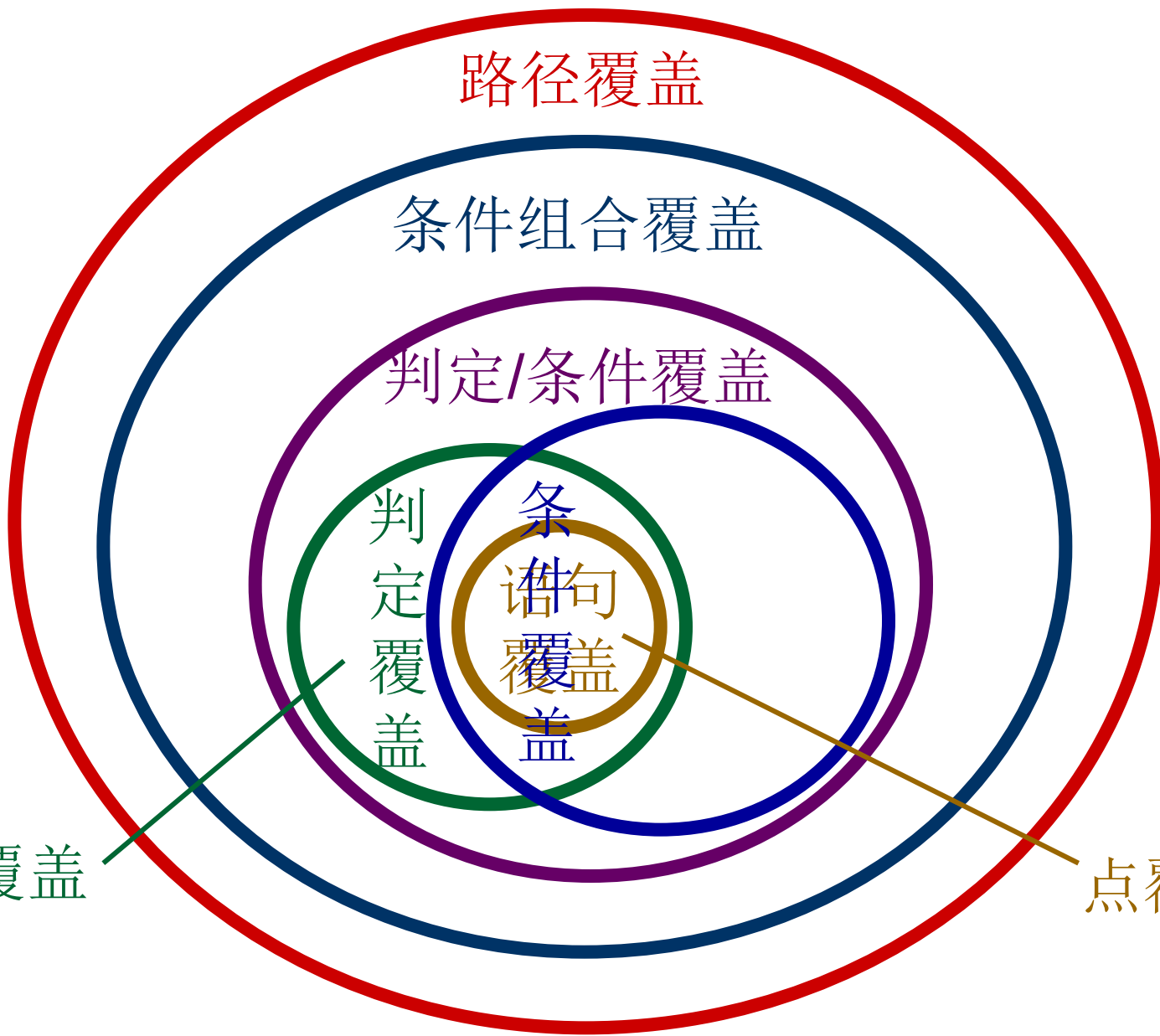
判定/条件覆盖

判定覆盖

语句覆盖
条件覆盖

边覆盖

点覆盖



总结： 6 种覆盖标准的对比

发现 错误 能力	弱	语句覆盖	每条语句至少执行一次
		判定覆盖	每个判定的每个分支至少执行一次
		条件覆盖	每个判定的每个条件应取到各种可能的值
		判定/条件覆盖	同时满足判定覆盖和条件覆盖
		条件组合覆盖	每个判定中各条件的每一种组合至少出现一次
	强	路径覆盖	使程序中每一条可能的路径至少执行一次

白盒测试例题

- 点覆盖测试用例

$A=2, X=3, Y=0$

- 边覆盖测试用例:

$A=2, X=3, Y=0$

$A=3, X=1, Y=1$

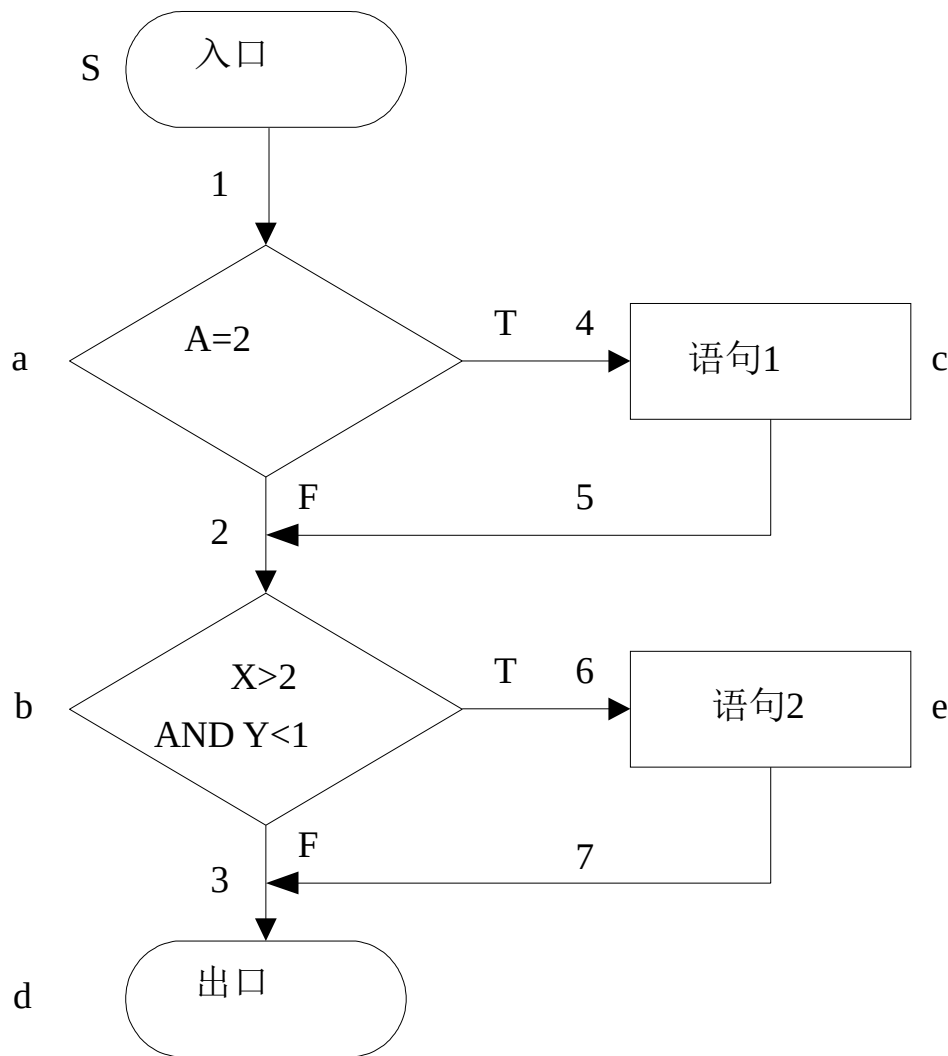
- 条件组合覆盖测试用例

$A=2, X=4, Y=-1$

$A=3, X=1, Y=2$

$A=2, X=4, Y=2$

$A=3, X=1, Y=-1$



4.4 黑盒测试技术

黑盒测试主要是为了发现以下错误：

- 是否有不正确或遗漏了的功能？
- 能否正确地接受输入？能否正确的输出结果？
- 性能上是否能够满足要求？
- 是否有初始化或终止性错误？

几种黑盒测试技术：

等价类划分、边界值分析、错误推测法、因果

图

4.4.1 等价划分

基本思想：

把所有可能的输入数据（包括有效或无效的），划分成若干数据类（等价类），然后从每个数据类中选取少数有代表性的数据做为测试用例。

这种方法完全不考虑程序的内部结构，只依据程序的规格说明来设计测试用例。

设计测试用例的步骤

步骤 1：划分等价类(列出等价类表)

等价类是指输入数据的子集合。在该子集合中，各输入数据对于发现程序中的错误都是等效的。

根据程序功能说明，确定有效和无效的等价类

步骤2：根据等价类设计测试用例

包括有效等价类和无效等价类的设计。

步骤 1：划分等价类

等价类分为：

- **有效等价类：**是指对于程序的规格说明来说，是合理的，有意义的输入数据构成的集合。
- **无效等价类：**是指对于程序的规格说明来说，是不合理的，无意义的输入数据构成的集合。

在设计测试用例时，要同时考虑有效等价类和无效等价类的设计！！

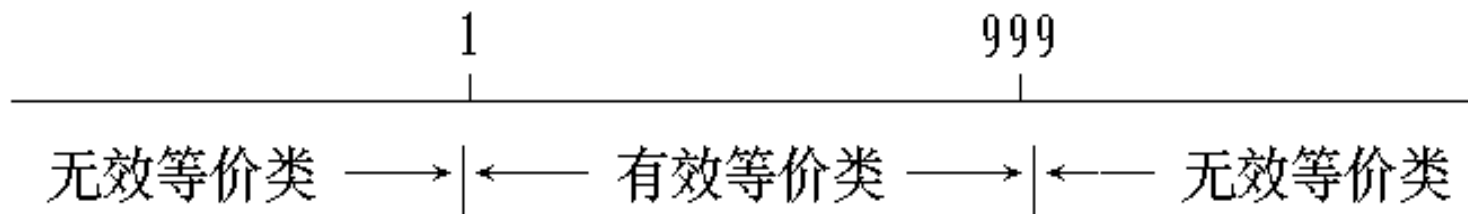
等价类划分原则（1）

原则1：若规定了取值范围，或输入值的个数，则可以确立一个有效等价类和两个无效等价类。

例：程序对输入条件的要求是：… 输入数是从 1 到 999 …

则，有效等价类是：“ $1 \leq \text{输入数} \leq 999$ ”；

两个无效等价类是：“输入数 < 1 ”或“输入数 > 999 ”
在数轴上表示成：



等价类划分原则（2）

原则2：如果规定了输入数据的一组值，而且程序要对每种输入数据分别处理，则可为每种输入值确立一个有效等价类，此外针对这组值确立一个无效等价类，它是所有不允许的输入值的集合。

例：教师上岗方案中规定对教授、副教授、讲师和助教分别计算分数，做相应的处理。因此可以确定4个有效等价类为教授、副教授、讲师和助教，一个无效等价类，它是所有不符合以上身分的人员的输入值的集合。

等价类划分原则（3）

原则 3：若规定了输入值的集合，或者是规定了“必须如何”的条件，则可确立一个有效等价类和一个无效等价类。

例：Pascal 语言对变量标识符规定必须“以字母打头”，则所有以字母打头的构成有效等价类，而不以字母打头的归于无效等价类。

等价类划分原则（4）

原则4：如果规定输入数据为整型，则可划分出正整、零和负整数三个有效类，其他数据为无效类

原则5：如果程序处理对象是表格，则应使用空表、含一项和多项的表。

原则6：如果确知，已划分的等价类中各元素在程序中的处理方式不同，则应将此等价类进一步划分成更小的等价类。

步骤2：确立测试用例

在确立了等价类之后，建立等价类表，列出所有划分出的等价类。

输入条件	有效等价类	无效等价类
***** *****	***** *****	***** *****

测试用例的选择原则

原则1: 为每一个等价类规定一个唯一编号;

原则2: 设计一个新的测试用例, 使其尽可能多地覆盖尚未被覆盖的有效等价类, 重复这一步, 直到所有的有效等价类都被覆盖为止;

原则3: 设计一个新的测试用例, 使其仅覆盖一个尚未被覆盖的无效等价类, 重复这一步, 直到所有的无效等价类都被覆盖为止。

等价类划分法设计测试用例（例 1）

例1:

某报表处理系统，要求用户输入处理报表的日期。

系统规定日期由年、月的 6 位数字字符组成，前 4 位代表年，后两位代表月。

设日期限制在 1990 年 1 月至 1999 年 12 月，即系统只能对该段时期内的报表进行处理。如果用户输入的日期不在此范围内，则显示输入错误。

现用等价类划分法设计测试用例，来测试程序的“日期检查功能”。

划分等价类并编号

输入条件	有效等价类	无效等价类
报表日期的类型及长度	① 6位数字字符	② 有非数字字符 ③ 少于6位数字字符 ④ 多余6位数字字符
年份范围	⑤ 在1990-1999之间	⑥ 小于1990 ⑦ 大于1999
月份范围	⑧ 在1-12之间	⑨ 等于0 ⑩ 大于12

为有效等价类设计测试用例

- ① 6 位数字字符
- ⑤ 在 1990 — 1999 之间
- ⑧ 在 1 — 12 之间

测试数据	期望结果	覆盖范围
199905	输入有效	1, 5, 8

为每个无效等价类设计测试用例

测试数据	期望结果	覆盖范围
1998A1	输入无效	②
19985	输入无效	③
1998005	输入无效	④
198912	输入无效	⑥
200001	输入无效	⑦
199900	输入无效	⑨
199913	输入无效	⑩

② 有非数字字符

③ 少于6位数字字符

④ 多于6位数字字符

⑥ 年份小于1990

⑦ 年份大于1999

⑨ 月份等于0

⑩ 月份大于12

等价类划分测试用例（例2）：

某城市的电话号码由3个部分组成：

地区码——空白或3位数字；

前缀——以大于等于‘5’开头的4位数字；

后缀——4位数字。

输入条件	有效等价类	无效等价类
地区码	(1) 空白 (2) 3位数字	(5) 有非数字字符 (6) 少于3位数字 (7) 多于3位数字
前缀	(3) 大于等于 ‘5’开头的4位数字	(8) 有非数字字符 (9) 少于4位数字 (10) 多于4位数字 (11) 开头数字小于 ‘5’
后缀	(4) 4位数字	(12) 有非数字字符 (13) 少于4位数字 (14) 多于4位数字

测试用例：略

等价类划分测试用例：

课堂测试题：某图书管理系统的图书录入功能要求为：图书名称不能为空且长度不能大于8个字符；图书价格必须在200元以内；图书出版日期必须为6位数字字符（其中，前4位数字表示年份，后2位数字表示月份）。图书录入数据的格式为：（图书名称，价格，出版日期）；例如：输入（西游记，50，201603）表示图书名称为“西游记”，价格为50元，出版日期为2016年3月。请使用等价类划分方法为该图书录入功能设计测试用例。

1) 划分等价类:

输入条件	有效等价类	无效等价类
图书名称	(1) 名称不能为空 且长度不能大于8个字符	(2) 名称为空 (3) 名称长度大于8个字符
图书价格	(4) 图书价格必须在200元以内	(5) 价格大于200 (6) 价格为负数
图书出版日期	(7) 出版日期必须为6位数字字符 (8) 前4位数字为年份 (9) 后2位数字为月份	(10) 日期长度小于6位 (11) 日期长度大于6位 (12) 日期为非数字 (13) 年份大于今年 (14) 月份大于12

2) 测试用例:

测试数据	预期结果	覆盖等价类
(西游记, 50, 201603)	有效	(1)、(4)、(7)、(8)、(9)
(, 50, 201603)	无效	(2)
(XXXXXXXXXX, 50, 201603)	无效	(3)
(西游记, 201, 201603)	无效	(5)
(西游记, -100, 201603)	无效	(6)
(西游记, 60, 20103)	无效	(10)
(西游记, 60, 20103110)	无效	(11)
(西游记, 60, 20103A)	无效	(12)
(西游记, 60, 202304)	无效	(13)
(西游记, 60, 201013)	无效	(14)

4.4.2 边界值分析

边界是指，对于输入和输出等价类而言，稍高和稍低于其边界值的一些特定情况。

经验得知，大量的错误是发生在输入或输出范围的边界上，而不是在输入范围的内部。

边界值分析方法思想：确定边界之后，选取正好等于、刚刚大于或刚刚小于边界的值做为测试数据，而不是选取等价类中典型值或任意值做为测试数据。

通常总是与等价划分技术联合使用，是等价划分方法的补充。

4.4.3 错误推测

- 不同类型不同特点的程序通常又有一些特殊的容易出错的情况。因此必须依靠测试人员的经验和直觉，从各种可能的测试方案中选出一些最可能引起程序出错的方案。
- 错误推测法在很大程度上靠直觉和经验进行。它的基本想法是列举出程序中可能有的错误和容易发生错误的特殊情况，并且根据它们选择测试方案。

4.5 .单元测试用例书

单元测试用例书